



Fit-Score Scoring & Matching Engine

Consolidated Technical Reference

Repository: backend/src/scoring

<https://github.com/COS301-SE-2026/ConsultIQ/tree/dev/backend/src/scoring>

Team: TROOS | Client: BBD | Compiled: September 4, 2026

Contents

1. Introduction & Purpose
 2. System Architecture
 3. Two-Tier Configuration & Weight Resolution
 4. Per-Factor Scoring Formulas
 5. Score Aggregation
 6. Full Worked Step-by-Step Example
 7. API Data Contracts
 8. End-to-End Match Run Flow
 9. Engineering Notes, Edge Cases & Onboarding Checklist
- Appendix A — Source Reconciliation Note

1. Introduction & Purpose

This document is the definitive reference for ConsultIQ's core feature: the fit-score matching engine. The engine evaluates and ranks consultants against a project's requirements, weighing skills, skill competency levels, cost, geographic proximity, and availability. It produces a transparent, auditable breakdown explaining exactly how each consultant's score was calculated.

The Scoring Engine serves three primary purposes:

- Scoring — compute a quantitative fit score for each consultant across five independent dimensions.
- Ranking — aggregate individual factor scores into a single composite total and rank consultants from best to worst fit.
- Mathematical transparency — output a structured payload alongside the final score, detailing the exact normalized weights and per-factor scores used, so the process is fully auditable.

Every factor score is normalized to a 0–1 scale before aggregation:

Score	Meaning
0	No fit, or invalid data for that factor.
0.5	Partial fit.
1	Full fit.

The weighted aggregator then converts the normalized per-factor scores into a final percentage:

$$finalScore = 100 \times \Sigma (factorScore_i \times weight_i), \text{ for } i = 1..n$$

Factor implementations live in `backend/src/scoring/services/five-scoring-modules`.

2. System Architecture

The engine runs as a four-stage processing pipeline. Each stage is an independent filter that takes in data, performs one task, and passes a refined output to the next stage — a modular design intended to maximize testability, auditability, and flexibility.

2.1 Pipeline Stages

1. Data Ingestion & Normalization — receives raw consultant and project data and converts it to a quantifiable 0.0–1.0 scale for uniform comparison.
2. Two-Tier Configuration Resolution — resolves consultancy-wide defaults against project-level overrides; active weights are normalized to sum to 1.0.
3. Scoring Processor (five modules) — runs the five independent scoring modules (Skill Alignment, Competency Match, Cost Fit, Geographic Fit, Availability) in parallel; each returns a score between 0.0 and 1.0 for every consultant in the pool.
4. Weighted Aggregator — multiplies each factor score by its normalized weight, sums the weighted contributions into a single final score, and sorts all consultants descending to produce the ranked list.

2.2 Components

Component	File	Responsibility
ScoringModule	scoring.module.ts	Registers controllers, orchestrators, factor scorers, normalization services, and aggregators.
ScoringController	scoring.controller.ts	REST endpoints to query and update default scoring config and project overrides.
MatchRunController	match-run.controller.ts	REST endpoints to trigger match runs and retrieve match run stats.
ScoringConfigService	scoring-config.service.ts	Two-tier configuration resolution (firm defaults vs. project overrides).
DataIngestionService	data-ingestion.service.ts	Prepares and normalizes raw consultant and project payloads before pipeline execution.
ScoringPipelineService	scoring-pipeline.service.ts	Coordinates end-to-end scoring pipeline execution.
ScoringOrchestrator	scoring.orchestrator.ts	Executes enabled scorers, enforces hard exclusions, calculates redistributed weights.
WeightedAggregator	weighted-aggregator.service.ts	Multiplies factor scores by redistributed weights to produce the composite score.

2.3 Data Flow Summary

Raw consultant and project data enters Data Ingestion & Normalization, which feeds the Scoring Processor. In parallel, the Two-Tier Configuration Resolver (Project-Level Config over Firm Default Config) determines which of the five scorers are active and their normalized weights. The five scorers — skill, competency, cost, location, and availability — each emit a 0.0–1.0 score, which the Weight Aggregator combines into the final composite score and ranked list.

3. Two-Tier Configuration & Weight Resolution

Scoring configuration is resolved dynamically across two tiers, stored as Prisma entities:

- Tier 1 — Global Defaults (ConsultancyScoringConfig): baseline firm-wide weights and factor toggle flags.
- Tier 2 — Project Overrides (ProjectScoringOverride): optional project-specific weight and toggle overrides, which always take precedence.

For each factor k in {SKILL_ALIGNMENT, COMPETENCY_LEVEL, COST_TO_COMPANY, LOCATION, AVAILABILITY}:

*$enabled_k = ProjectOverride.enabled_k ?? FirmDefault.enabled_k$
 $weight_k = ProjectOverride.weight_k ?? FirmDefault.weight_k$*

A factor is active only if $enabled_k = true$ and $weight_k > 0$.

3.1 Resolution Rules

1. Load all five factors from ConsultancyScoringConfig as the baseline.
2. For each factor, check whether a ProjectScoringOverride record exists for the current project.
3. If an override exists, replace the baseline weight and is_active with the override values.

4. Remove all inactive factors from the resolved set.
5. Normalize remaining weights by dividing each by the sum of all remaining raw weights.
6. Assert the normalized weights sum to exactly 1.0 before passing to the scoring processor.
7. Division-by-zero safeguard: if every factor is inactive, or the sum of raw active weights is 0, catch the exception and fall back to an equal 0.20 distribution across all five baseline factors.
8. ProjectScoringOverride always takes precedence over ConsultancyScoringConfig for a given placement run.

3.2 Weight Normalization Formula

$$w_normalized[i] = w_raw[i] / \sum(j \in active) w_raw[j]$$

3.3 Weight Redistribution (Orchestrator Level)

When a subset of factors is active, the weight belonging to inactive factors is proportionally redistributed across the remaining active factors:

$$W_redistributed(a) = W_a + (W_a / W_active_sum) \times W_inactive_sum$$

Note: If $W_active_sum \leq 0$ or all factors are disabled, the function throws an `InternalServerErrorException` rather than silently returning zeros — callers must treat this as a hard failure, not a degraded result. (The configuration resolver's own division-by-zero safeguard in 3.1 step 7 covers the equal-distribution fallback at the config layer; the orchestrator-level redistribution in this section is stricter and fails loudly.)

4. Per-Factor Scoring Formulas

The five scorers below are the authoritative, code-level calculations (from the scoring-factor calculation reference). Each returns a value in the 0.0–1.0 range unless noted, along with a human-readable `factorDetails` explanation.

4.1 Skill Alignment — SkillAlignmentScorer

Inputs: project required skill names, whether each is mandatory, and consultant skill names. Skill names are trimmed and compared case-insensitively. Competency levels are not considered here — they belong to the Competency factor.

$$skillAlignmentScore = M / R \text{ (clamped to } [0, 1])$$

Where M is the number of required skills the consultant possesses, and R is the total number of project-required skills.

Required skills	Possessed	Calculation	Score
4	4	4 / 4	1.00
4	3	3 / 4	0.75
4	0	0 / 4	0.00

Hard exclusion

If one or more mandatory skills are missing, the scorer sets `triggerHardExclusion = true` and returns the missing skill names. The orchestrator then applies configuration:

- Hard exclusion enabled: the consultant is removed from the result set entirely.
- Hard exclusion disabled: the consultant remains eligible, but 0.15 is subtracted from the skill score:

$$\text{penalizedSkillScore} = \max(0, \text{skillAlignmentScore} - 0.15)$$

Example: a raw score of 0.75 becomes 0.60 when mandatory skills are missing and hard exclusion is disabled.

Note: If the project has no required skills at all, the scorer returns score 0, flags the result as an exclusion candidate, and reports invalid project data — though match-run validation normally rejects such projects earlier, before scoring.

4.2 Competency Level — CompetencyMatchScorer

Each required skill is scored by comparing the consultant's competency rank against the required rank.

Level	Rank
Missing	0.0
Beginner	0.3
Intermediate	0.6
Expert	1.0

Requirement type	Weight
Mandatory skill	2.0
Optional skill	1.0

Mandatory requirements contribute twice as much as optional requirements to the competency score.

Per-skill score

Let C be the consultant's competency rank, Q the required rank, and $D = C - Q$ the rank difference.

Condition	Per-skill score
Consultant does not possess the skill	0
Required level missing/invalid, consultant has the skill	1
Consultant exactly meets the requirement ($D = 0$)	1.0
Consultant is below requirement ($D < 0$)	C / Q
Consultant is above requirement ($D > 0$)	$\max(0.7, 1.0 - (D \times 0.15))$

Note: The over-qualification penalty prevents a higher competency from automatically scoring a perfect 1.0, but the per-skill score can never fall below 0.70 purely from being over-qualified. The implementation uses competency ordinal differences for this step, so the practical penalty is based on the number of discrete levels above the requirement.

Weighted base score, extra-skill bonus, and final score

$$\text{baseCompetencyScore} = \Sigma(S_i \times W_i) / \Sigma(W_i)$$

The consultant also receives a small bonus for skills they hold that were not matched to any project requirement:

$$\begin{aligned} \text{extraSkillBonus} &= 0.02 \times \text{extraSkillCount} \\ \text{competencyScore} &= \min(1, \text{baseCompetencyScore} + \text{extraSkillBonus}) \end{aligned}$$

Worked examples

- Exact mandatory match — one mandatory Intermediate requirement, consultant at Intermediate: per-skill score 1.0, weight 2.0 → base score $(1.0 \times 2.0) / 2.0 = 1.0$; no extra skills → final score 1.0.
- Consultant below requirement — one mandatory Expert requirement, consultant at Intermediate: $0.6 / 1.0 = 0.6$ → final score 0.6.
- Consultant above requirement — one mandatory Beginner requirement, consultant at Expert: rank difference $1.0 - 0.3 = 0.7$ → $1.0 - (0.7 \times 0.15) = 0.895$ → final score 0.895.

Note: If no competency levels are required, this factor returns 1.0. This factor never triggers a hard exclusion.

4.3 Cost to Company — CostFitScorer

Inputs: consultant costToCompany and project billingBudgetPerHour.

Within budget

$$\text{cost} \leq \text{budget} \Rightarrow \text{costScore} = 1.0$$

Example: cost 400 against budget 500 → score 1.0.

Over budget

$$\begin{aligned} \text{overage} &= (\text{cost} - \text{budget}) / \text{budget} \\ \text{penalty} &= (\text{overage} / 0.3) \times 0.2 \\ \text{costScore} &= \max(0, 1 - \text{penalty}) \end{aligned}$$

The reference decrease rate is 0.3 and the penalty factor is 0.2 (in code, the constant is named *decreaseRate* — the misspelling is intentional to keep as-is when referencing the source).

Cost	Budget	Overage	Penalty	Score
650	500	$(650 - 500) / 500 = 0.30$	$(0.30 / 0.30) \times 0.20 = 0.20$	0.80
800	500	$(800 - 500) / 500 = 0.60$	$(0.60 / 0.30) \times 0.20 = 0.40$	0.60

This penalty is linear with no upper overage cap — at sufficiently high cost, the score reaches zero.

Note: If either cost or budget is missing, zero, or negative, the factor returns 0 with a detail explaining that the financial data is invalid. It does not trigger a hard exclusion.

4.4 Geographic Fit — GeographicFitScorer

The scorer prefers coordinate-based travel data and falls back to city/province string matching when coordinates or travel data are unavailable. Coordinates are used only when both consultant and project latitude/longitude values are truthy — a coordinate value of 0 is treated as missing, since the implementation uses truthiness checks.

Coordinate-based calculation

The location service returns `distanceMeters` and `durationSeconds` (both must be finite numbers; invalid or failed responses fall back to the string method).

Duration score — used when travel duration is positive:

$$\begin{aligned} \text{durationMinutes} &= \text{durationSeconds} / 60 \\ \text{locationScore} &= \max(0.1, e^{(-\text{durationMinutes} / 45)}) \end{aligned}$$

Travel time	Score
45 minutes	≈ 0.368
90 minutes	≈ 0.135
Very long	0.10 (floor)

Distance score — used when duration is zero:

$$\begin{aligned} \text{distanceKm} &= \text{distanceMeters} / 1000 \\ \text{locationScore} &= \max(0.1, e^{(-\text{distanceKm} / 40)}) \end{aligned}$$

Distance	Score
0 km	1.00
40 km	≈ 0.368
80 km	≈ 0.135
Very long	0.10 (floor)

String fallback calculation

City and province values are trimmed and compared case-insensitively; the fallback assigns a fixed score rather than calculating an actual distance:

Condition	Score
Same city, usable city values	0.8
Different city, same province	0.5
Different province or no matching values	0.1

Data source & caching

The factor records where its result came from: `api-duration` (valid metrics, positive duration), `api-distance` (valid metrics, zero duration), or `fallback` (missing coordinates, API failure, or malformed metrics).

- Travel requests are cached using a key built from coordinates rounded to two decimal places.
- Successful metrics: 24-hour TTL. Failed/malformed requests: 5-minute negative-cache TTL.
- Maximum 5,000 cache entries; identical simultaneous requests share one pending promise.

Note: Geographic Fit never triggers a hard exclusion.

4.5 Availability — AvailabilityFitScorer

Inputs: the consultant's overlapping placement allocations, the project's start/end dates, and the project's required allocation percentage.

Overlap rules

A placement is included in the overlap calculation when:

1. Its status is not TERMINATED or CANCELLED.
2. Its start date is on or before the project end date (when the project has one).
3. Its end date is on or after the project start date, or its end date is null (open-ended placements still count against capacity via an explicit OR clause).

The allocation query sums all matching placement allocation percentages (selected as allocation, not allocationPercentage).

Remaining capacity & score

$$\text{remainingCapacity} = 100 - \text{totalOverlappingAllocation}$$

Note: The consultant's own capacity database field is not used in this calculation — only the sum of overlapping placement allocations.

Let R be the project's required allocation percentage:

Condition	Score
$R \leq 0$	1.0
$\text{remainingCapacity} \leq 0$	0.0 (also triggers hard-exclusion evaluation)
$\text{remainingCapacity} \geq R$	1.0
Otherwise	$\text{remainingCapacity} / R$

- Enough capacity — 30% existing allocation → 70% remaining; 50% required → score 1.0.
- Partial capacity — 75% existing allocation → 25% remaining; 50% required → score $25/50 = 0.5$.
- No capacity — 100% existing allocation → 0% remaining → score 0.0.

Note: When remaining capacity is zero or less, the scorer sets `triggerHardExclusion = true`. With hard exclusion enabled the consultant is removed; with it disabled the consultant remains eligible with an availability score of 0.

4.6 Factor Summary

Factor	Score range	Hard-exclusion?	Core calculation
Skill Alignment	0 – 1	Yes	Matched required skills ÷ total required skills.
Competency Level	0 – 1	No	Weighted per-skill competency comparison + extra-skill bonus.
Cost to Company	0 – 1	No	Full score in budget; linear over-budget penalty.

Factor	Score range	Hard-exclusion?	Core calculation
Geographic Fit	0.1 – 1	No	Exponential travel-time/distance decay, or city/province fallback.
Availability	0 – 1	Yes	Remaining capacity ÷ required allocation when partially available.

Note: Hard exclusion is enforced only when the factor is active and its `hardExclusionEnabled` configuration flag is true.

5. Score Aggregation

The Weighted Aggregator runs once all five factor scores are available for a consultant. It implements a Weighted Sum Model (WSM) — the standard approach in multi-criteria decision analysis for ranking alternatives across independent dimensions.

- Multiplies each factor score by its redistributed/normalized weight.
- Sums the weighted contributions into a composite score between 0.0 and 1.0.
- Sorts all consultants in the pool descending by composite score and assigns rank 1 to the highest scorer.
- Passes the ranked list to the Explainability Generator.

$$\text{compositeScore} = \sum (\text{factorScore}_a \times W_{\text{redistributed}}(a)), \text{ for } a \text{ in active factors}$$

$$\text{finalScore (displayed)} = 100 \times \text{compositeScore}$$

Factor raw scores, weights, and contributions are rounded to four decimal places in the persisted breakdown; the displayed final score is rounded to two decimal places.

5.1 Illustrative Weighting Example

Factor	Raw score	Weight	Weighted contribution
Skill alignment	0.75	0.40	0.3000
Competency level	0.80	0.30	0.2400
Cost to company	1.00	0.15	0.1500
Location	0.50	0.10	0.0500
Availability	1.00	0.05	0.0500

$$\text{finalScore} = 100 \times (0.3000 + 0.2400 + 0.1500 + 0.0500 + 0.0500) = 79.00$$

6. Full Worked Step-by-Step Example

6.1 Context

- Project skills required: NestJS (Expert, Mandatory), PostgreSQL (Intermediate, Optional).
- Budget: \$100/hr | Locality: Pretoria, Gauteng | Required allocation: 20%.

- Active weights: Skill Alignment 0.35, Competency Level 0.25, Cost Fit 0.15, Availability 0.15, Location 0.10 (Location is disabled/inactive for this project).
- Candidate skills: NestJS (Expert), PostgreSQL (Intermediate), Docker (Intermediate). Rate: \$110/hr (10% over budget). Locality: Johannesburg, Gauteng. Remaining capacity: 20%.

6.2 Step 1 — Weight Redistribution

Inactive sum = 0.10 (Location). Active sum = 0.90.

Factor	Redistributed weight
Skill Alignment	$0.35 + (0.35 / 0.90) \times 0.10 = 0.3889$
Competency Level	$0.25 + (0.25 / 0.90) \times 0.10 = 0.2778$
Cost Fit	$0.15 + (0.15 / 0.90) \times 0.10 = 0.1667$
Availability Fit	$0.15 + (0.15 / 0.90) \times 0.10 = 0.1667$

6.3 Step 2 — Per-Factor Scoring

- Skill Alignment — possesses 2 of 2 required skills → score 1.0; mandatory skill (NestJS) present, no exclusion.
- Competency, NestJS — required Expert (1.0), has Expert (1.0) → per-skill score 1.0, weight 2.0 (mandatory).
- Competency, PostgreSQL — required Intermediate (0.6), has Intermediate (0.6) → per-skill score 1.0, weight 1.0 (optional).
- Competency base score = $(1.0 \times 2.0 + 1.0 \times 1.0) / 3.0 = 1.0$; extra-skill bonus = 1 extra skill (Docker) $\times 0.02 = 0.02$ → final = $\min(1.0, 1.02) = 1.0$.
- Cost Fit — cost \$110 > budget \$100 → overage = $(110 - 100) / 100 = 0.10$; penalty = $(0.10 / 0.3) \times 0.2 = 0.0667$ → score = $1.0 - 0.0667 = 0.9333$.
- Availability Fit — 20% remaining capacity against a 20% requirement → score = $20 / 20 = 1.0$.

6.4 Step 3 — Composite Score

$$\begin{aligned} \text{compositeScore} &= (1.0 \times 0.3889) + (1.0 \times 0.2778) + (0.9333 \times 0.1667) + (1.0 \times 0.1667) \\ &= 0.3889 + 0.2778 + 0.1556 + 0.1667 = 0.9890 \text{ (98.90\%)} \end{aligned}$$

7. API Data Contracts

7.1 Standard Successful Match Payload

```
{
  "consultantId": "c8d32b84-482f-4c91-92e1-88432a1012ab",
  "projectId": "p92f81a1-120a-431b-8012-91f822a109cc",
  "excluded": false,
  "compositeScore": 0.9890,
  "factorScores": {
    "SKILL_ALIGNMENT": 1.0,
    "COMPETENCY_LEVEL": 1.0,
    "COST_TO_COMPANY": 0.9333,
    "AVAILABILITY": 1.0
  }
},
```

```

"redistributedWeights": {
  "SKILL_ALIGNMENT": 0.3889,
  "COMPETENCY_LEVEL": 0.2778,
  "COST_TO_COMPANY": 0.1667,
  "AVAILABILITY": 0.1667
},
"factorDetails": {
  "SKILL_ALIGNMENT": "Matched 2 of 2 required skills (Missing: none)",
  "COMPETENCY_LEVEL": "Base competency 100.0%; +0.02 extra skill bonus (1 extra skill)",
  "COST_TO_COMPANY": "10.0% over budget ($110 vs $100); Penalty applied: 0.0667",
  "AVAILABILITY": "Requires 20% capacity | Has 20% remaining"
}
}

```

7.2 Hard Exclusion Response Payload

```

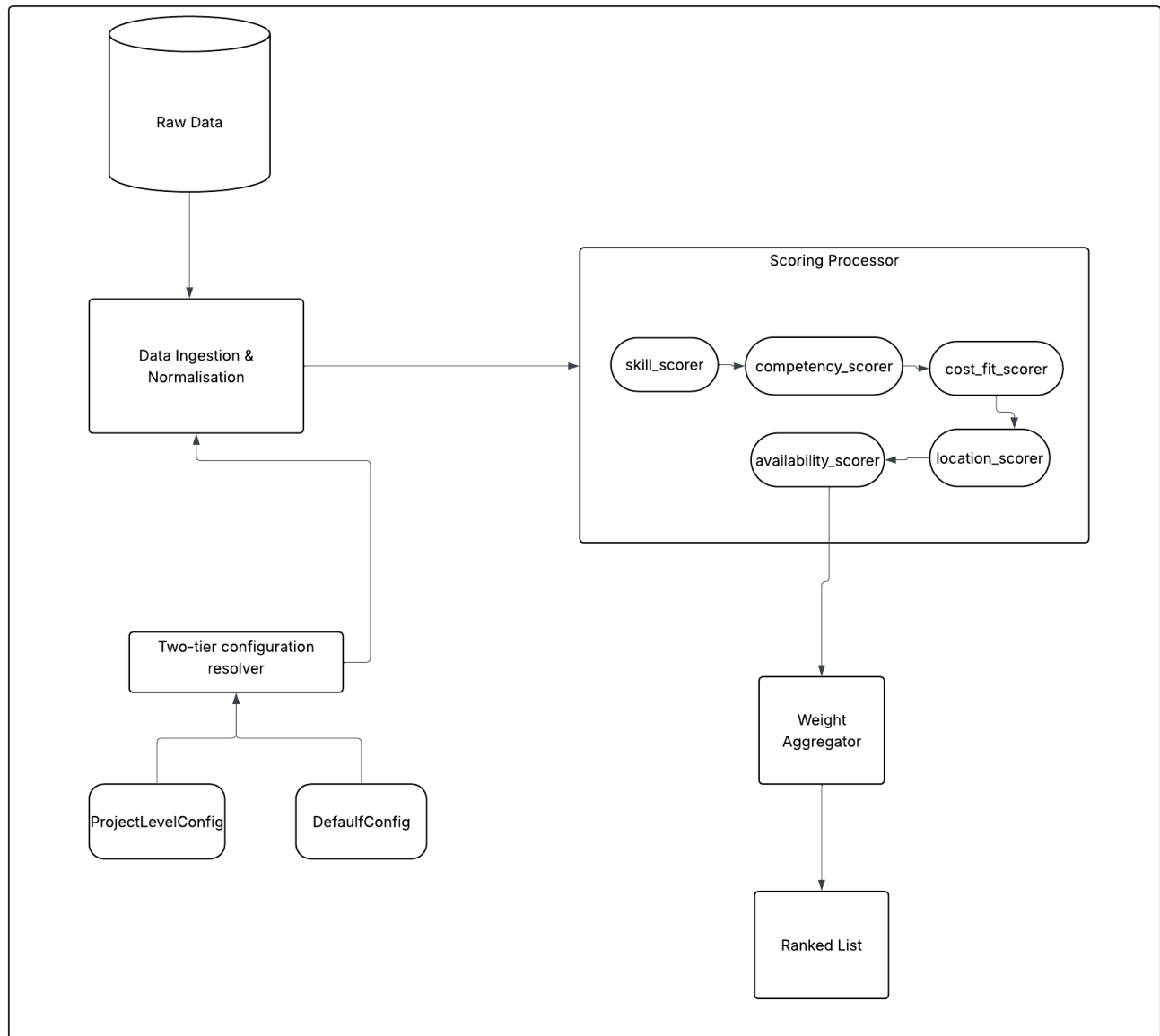
{
  "consultantId": "c4109f22-921a-4221-a182-1200921ab012",
  "projectId": "p92f81a1-120a-431b-8012-91f822a109cc",
  "excluded": true,
  "compositeScore": 0.0,
  "reason": "Missing mandatory required skill(s): NestJS",
  "missingMandatorySkills": ["NestJS"]
}

```

8. End-to-End Match Run Flow & Component Architecture Diagram

The following sequence describes the complete execution path for a single match run triggered by a Project Manager.

Step	Action
START	Project Manager triggers a match run for Project X via the dashboard.
1	System validates the project status is Open. Projects with status In Progress or Closed return a clear error immediately.
2	Fetch all consultant profiles.
3	Fetch the full project specification for Project X.
4	Load ConsultancyScoringConfig (firm-wide defaults); apply ProjectScoringOverride if one exists; normalize resolved weights to sum to 1.0.
5	For each consultant in the interested pool, run all five scoring modules in parallel; apply mandatory-skill hard exclusion if active.
6	Run the Weighted Aggregator: $\text{finalScore} = \sum(\text{weight}_i \times \text{score}_i)$ for active factors; sort consultants descending; assign ranks.
7	Run the Explainability Generator for each result: plain-language reasons, positive/negative factors, an improvement hint, structured as a JSON payload.
8	Store the match run record with a configuration snapshot (full weights, active states, hard-exclusion setting at time of execution) and individual consultant results.
END	Return the ranked list with full explainability payloads to the Placement Recommendation Dashboard.



9. Engineering Notes, Edge Cases & Onboarding Checklist

9.1 Onboarding Checklist

- Short-circuit vs. full run — a hard exclusion (missing mandatory skill or zero availability) aborts scoring early to save compute.

- Weight integrity — weights are normalized dynamically via redistribution so active factors always sum to 1.0.
- Database auditability — factor details and scores are persisted in Prisma MatchRunResult records for debugging and dashboard visualization.

9.2 Edge Cases to Watch

- Skill name matching is simple normalization (trim + lowercase); synonyms and aliases will not match unless pre-normalized during ingestion.
- Competency ranks are fractional, not ordinal integers — relevant if new competency levels are ever added.
- The overqualification floor (0.70) means even a strongly overqualified consultant scores at least 0.70 on that skill — intentional, but worth a business/product review.
- Weight redistribution throws rather than silently degrading when all factors are disabled or total active weight is 0 (see 3.3).
- Availability scoring depends entirely on accurate projectPlacement records — including correctly handling open-ended placements — and correct date-overlap logic; a bug here silently affects match quality.
- Coordinate values of 0 are treated as missing by the Geographic Fit scorer because the implementation uses truthiness checks.

9.3 Where to Look Next

- Data ingestion & normalization: `data-ingestion.service.ts` and `normalization.service.ts` — define the consultant/project shapes and how config is resolved.
- Scoring config persistence & overrides: `scoring-config.service.ts` (see `resolveProjectWeights` and its integration test for the two-tier fallback logic).
- Match-run persistence/aggregation: `MatchRunService` and `MatchRunAggregationService` — how results are stored/consumed and how `WeightedAggregator` combines factor scores into a final fit score.
- Tests: spec files (`scoring.orchestrator.spec.ts`, `scoring-pipeline.service.spec.ts`, etc.) are the best source of truth for expected behavior and edge cases.